Passing and Returning Objects in Java

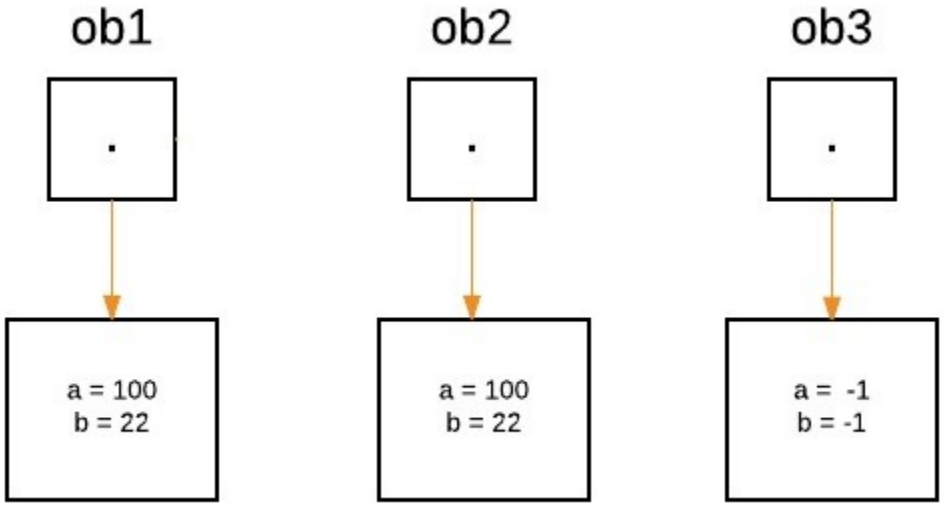
Last Updated : 27 Dec, 2021

Although Java is strictly passed by value, the precise effect differs between whether a primitive type or a reference type is passed. When we pass a primitive type to a method, it is passed by value. But when we pass an object to a method, the situation changes dramatically, because objects are passed by what is effectively call-by-reference. Java does this interesting thing that's sort of a hybrid between pass-by-value and pass-by-reference.
Basically, a parameter cannot be changed by the function, but the function can ask the parameter to change itself via calling some method within it.

- While creating a variable of a class type, we only create a reference to an object. Thus, when we pass this reference to a method, the parameter that receives it will refer to the same object as that referred to by the argument.
- This effectively means that objects act as if they are passed to methods by use of call-by-reference.
- Changes to the object inside the method do reflect the object used as an argument.

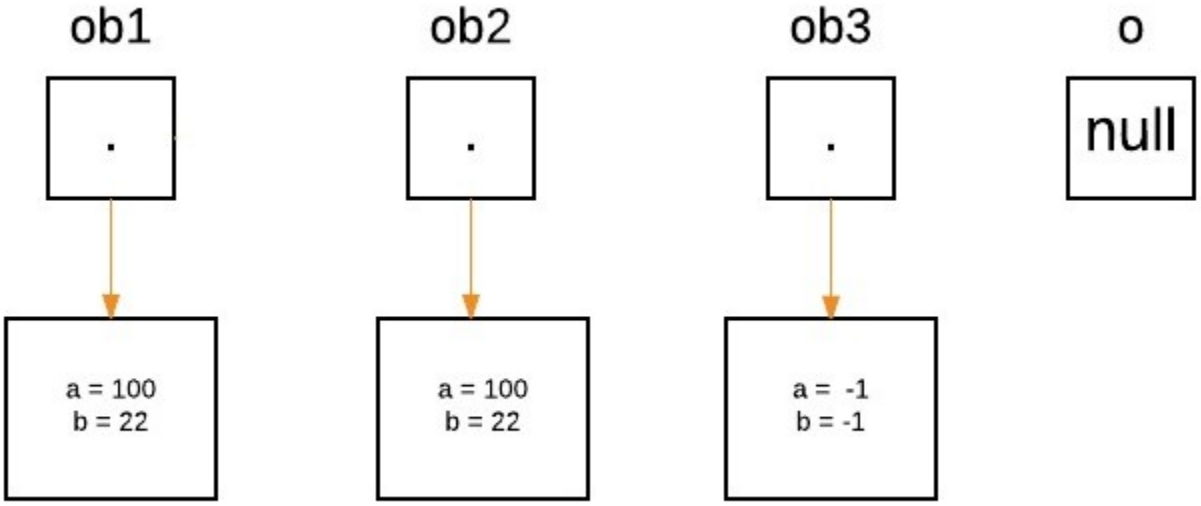
Illustration: Let us suppose three objects 'ob1' , 'ob2' and 'ob3' are created:

```
ObjectPassDemo ob1 = new ObjectPassDemo(100, 22);
ObjectPassDemo ob2 = new ObjectPassDemo(100, 22);
ObjectPassDemo ob3 = new ObjectPassDemo(-1, -1);
```



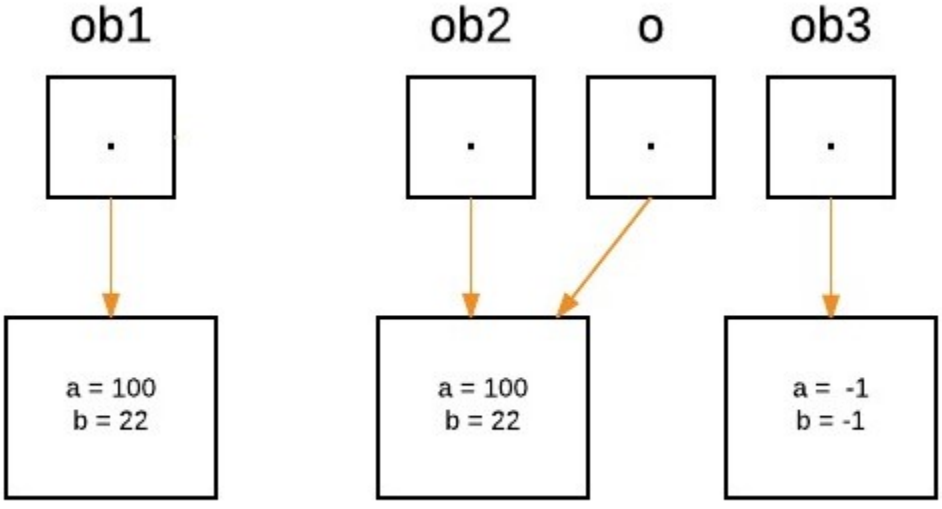
From the method side, a reference of type Foo with a name a is declared and it's initially assigned to null.

```
boolean equalTo(ObjectPassDemo o);
```



As we call the method equalTo, the reference 'o' will be assigned to the object which is passed as an argument, i.e. 'o' will refer to 'ob2' as the following statement execute.

```
System.out.println("ob1 == ob2: " + ob1.equalTo(ob2));
```

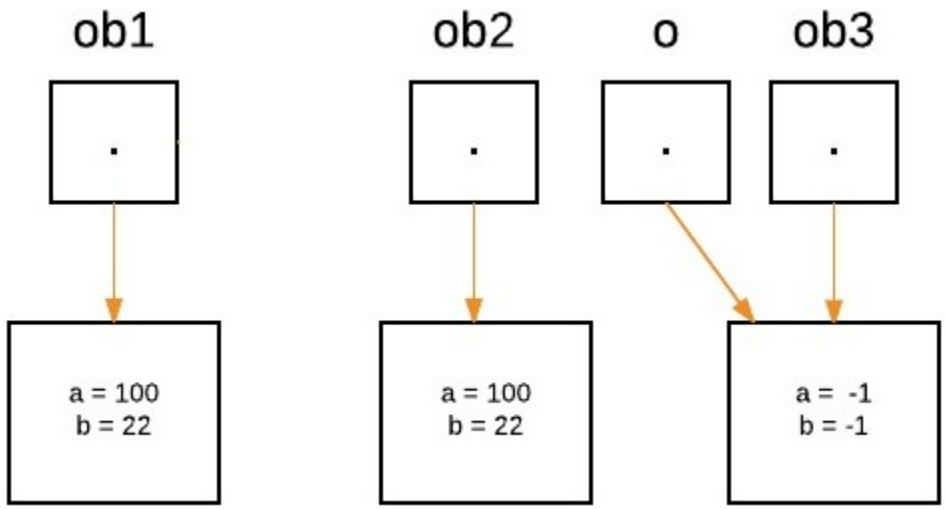


Now as we can see, equalTo method is called on 'ob1' , and 'o' is referring to 'ob2'. Since values of 'a' and 'b' are same for both the references, so if(condition) is true, so boolean true will be return.

```
if(o.a == a && o.b == b)
```

Again ‘o’ will reassign to ‘ob3’ as the following statement execute.

```
System.out.println("ob1 == ob3: " + ob1.equalTo(ob3));
```



- Now as we can see, the equalTo method is called on ‘ob1’ , and ‘o’ is referring to ‘ob3’. Since values of ‘a’ and ‘b’ are not the same for both the references, so if(condition) is false, so else block will execute, and false will be returned.

In Java we can pass objects to methods as one can perceive from the below program as follows:

Example:

```
// Java Program to Demonstrate Objects Passing to Methods.

// Class
// Helper class
class ObjectPassDemo {
    int a, b;

    // Constructor
    ObjectPassDemo(int i, int j)
    {
        a = i;
        b = j;
    }

    // Method
```

```
    boolean equalTo(ObjectPassDemo o)
    {
        // Returns true if o is equal to the invoking
        // object notice an object is passed as an
        // argument to method
        return (o.a == a && o.b == b);
    }
}

// Main class
public class GFG {
    // MAin driver method
    public static void main(String args[])
    {
        // Creating object of above class inside main()
        ObjectPassDemo ob1 = new ObjectPassDemo(100, 22);
        ObjectPassDemo ob2 = new ObjectPassDemo(100, 22);
        ObjectPassDemo ob3 = new ObjectPassDemo(-1, -1);

        // Checking whether object are equal as custom
        // values
        // above passed and printing corresponding boolean
        // value
        System.out.println("ob1 == ob2: "
                           + ob1.equalTo(ob2));
        System.out.println("ob1 == ob3: "
                           + ob1.equalTo(ob3));
    }
}
```

Output

```
ob1 == ob2: true
ob1 == ob3: false
```

Defining a constructor that takes an object of its class as a parameter

One of the most common uses of object parameters involves constructors. Frequently, in practice, there is a need to construct a new object so that it is initially the same as some existing object. To do this, either we can use Object.clone() method or define a constructor that takes an object of its class as a parameter.

Example

```
// Java program to Demonstrate One Object to
// Initialize Another

// Class 1
class Box {
    double width, height, depth;

    // Notice this constructor. It takes an
    // object of type Box. This constructor use
    // one object to initialize another
    Box(Box ob)
    {
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }

    // constructor used when all dimensions
    // specified
    Box(double w, double h, double d)
    {
        width = w;
        height = h;
        depth = d;
    }

    // compute and return volume
    double volume() { return width * height * depth; }
}

// MAin class
public class GFG {
    // Main driver method
    public static void main(String args[])
    {
        // Creating a box with all dimensions specified
        Box mybox = new Box(10, 20, 15);

        //  Creating a copy of mybox
        Box myclone = new Box(mybox);

        double vol;

        // Get volume of mybox
```

```
        vol = mybox.volume();
        System.out.println("Volume of mybox is " + vol);

        // Get volume of myclone
        vol = myclone.volume();
        System.out.println("Volume of myclone is " + vol);
    }
}
```

Output

```
Volume of mybox is 3000.0
Volume of myclone is 3000.0
```

Returning Objects

In java, a method can return any type of data, including objects. For example, in the following program, the **incrByTen( )** method returns an object in which the value of an (an integer variable) is ten greater than it is in the invoking object.

Example

```
// Java Program to Demonstrate Returning of Objects

// Class 1
class ObjectReturnDemo {
    int a;

    // Constructor
    ObjectReturnDemo(int i) { a = i; }

    // Method returns an object
    ObjectReturnDemo incrByTen()
    {
        ObjectReturnDemo temp
            = new ObjectReturnDemo(a + 10);
        return temp;
    }
}

// Class 2
```

```java
// Main class
public class GFG {

    // Main driver method
    public static void main(String args[])
    {

        // Creating object of class1 inside main() method
        ObjectReturnDemo ob1 = new ObjectReturnDemo(2);
        ObjectReturnDemo ob2;

        ob2 = ob1.incrByTen();

        System.out.println("ob1.a: " + ob1.a);
        System.out.println("ob2.a: " + ob2.a);
    }
}
```

Output

```
ob1.a: 2
ob2.a: 12
```

Note: *When an object reference is passed to a method, the reference itself is passed by use of call-by-value. However, since the value being passed refers to an object, the copy of that value will still refer to the same object that its corresponding argument does. That's why we said that java is strictly pass-by-value.*

Classes and Objects in Java

In Java, classes and objects are basic concepts of Object Oriented Programming (OOPs) that are used to represent real-world concepts and entities. The class represents a group of objects having similar...

11 min read

Understanding Classes and Objects in Java

The term Object-Oriented explains the concept of organizing the software as a combination of different types of objects that incorporate both data and behavior. Hence, Object-oriented programming(OOPs) is...

10 min read

Inner Class in Java

In Java, inner class refers to the class that is declared inside class or interface which were mainly introduced, to sum up, same logically relatable classes as Java is object-oriented so bringing it closer to...

11 min read

Anonymous Inner Class in Java

Nested Classes in Java is prerequisite required before adhering forward to grasp about anonymous Inner class. It is an inner class without a name and for which only a single object is created. An anonymous...

7 min read

Nested Classes in Java

In Java, it is possible to define a class within another class, such classes are known as nested classes. They enable you to logically group classes that are only used in one place, thus this increases the use of...

5 min read

Java.util.Objects class in Java

Java 7 has come up with a new class Objects that have 9 static utility methods for operating on objects. These utilities include null-safe methods for computing the hash code of an object, returning a string for...

7 min read

Different ways to create objects in Java

There are several ways by which we can create objects of a class in java as we all know a class provides the blueprint for objects, you create an object from a class. This concept is under-rated and sometimes...

6 min read

How are Java objects stored in memory?

In Java, all objects are dynamically allocated on Heap. This is different from C++ where objects can be allocated memory either on Stack or on Heap. In JAVA , when we allocate the object using new(), the...

3 min read

Passing and Returning Objects in Java

Although Java is strictly passed by value, the precise effect differs between whether a primitive type or a reference type is passed. When we pass a primitive type to a method, it is passed by value. But when w...

6 min read

Lambda Expression in Java

Lambda expressions in Java, introduced in Java SE 8, represent instances of functional interfaces (interfaces with a single abstract method). They provide a concise way to express instances of single-...

5 min read

Serialization and Deserialization in Java with Example

Serialization is a mechanism of converting the state of an object into a byte stream. Deserialization is the reverse process where the byte stream is used to recreate the actual Java object in memory. This...

7 min read

Garbage Collection in Java

Garbage collection in Java is the process by which Java programs perform automatic memory management. Java programs compile to bytecode that can be run on a Java Virtual Machine, or JVM for...

9 min read

How to prevent objects of a class from Garbage Collection in Java

The garbage collector in Java is automatic, i.e the user doesn't have to manually free an occupied memory which was dynamically allocated. And how does a garbage collector decide which object is to be...

5 min read

Count number of a class objects created in Java

The idea is to use static member in the class to count objects. A static member is shared by all objects of the class, all static data is initialized to zero when the first object is created if no other initialization is...

1 min read

Article Tags : Java School Programming

Practice Tags : Java